

የኢንፎርሜሽን መረብ ደህንነት አስተዳደር

Information Network Security Administration

DEVELOPMENT DEPARTMENT

5TH BATCH

NAME : HELEN TESFAYE

ID : CTC-1586-26

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1. Background of the Organization

1.2. General Description of Existing System

1.3. Problems of Existing System

1.4. Objective of the Project

1.4.1. General Objective

1.4.2. Specific Objectives

1.5. Overview of the Proposed System

1.6. Feasibility Analysis

1.6.1. Economic Feasibility

1.6.2. Technical Feasibility

1.6.3. Operational Feasibility

1.7. Relevance of the Proposed System

1.8. Scope of the Proposed System

CHAPTER 2: CONTEXTUAL ANALYSIS AND METHODOLOGICAL FRAMEWORK

2.1. Literature Review

2.1.1. Experiences of Similar but Failed Projects

2.1.2. Experiences of Successful Projects

2.1.3. Benchmarking Considerations

2.2. Methodology

2.2.1. Data/Requirement Gathering Methodology

2.2.2. Analysis and Modeling Methodology

2.2.3. Software Tools to Be Used

CHAPTER 3: REQUIREMENT ANALYSIS AND SPECIFICATION

3.1. Requirement Analysis

3.1.1. Analysis Approach (BPA/BPI/BPR)

3.1.2. Functional Requirements

3.1.3. Nonfunctional Requirements

3.2. Comparative Analysis and Solution Proposal

3.2.1. Benchmarking Analysis

3.2.2. Gap Analysis

3.2.3. Proposed Solution Based on Requirements

3.3. Requirement Modeling

3.3.1. Structured Requirement Modeling

3.3.2. Object-Oriented Requirement Modeling

CHAPTER 4: SYSTEM DESIGN

4.1. System Architecture

4.2. Interface Design

4.3. Database/Data Structure Design

4.4. Algorithm/Program Design

CHAPTER 5: RISK MITIGATION STRATEGY

5.1. Risk Identification

5.2. Risk Assessment

5.3. Mitigation Strategies

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1. Conclusion

6.2. Recommendations

6.3. Future Enhancements

6.4. Plan for System Construction, Testing, Conversion, and Support

REFERENCES

CHAPTER 1: INTRODUCTION

Access to blood during medical emergencies is an important challenge in healthcare. Delays may occur due to limited blood availability, difficulties in finding compatible blood types, and poor coordination between healthcare institutions and potential donors. MEKAKEL (መካከል) is a web-based blood coordination platform designed to connect hospitals, clinics, and volunteer blood donors to make blood requests and coordination more efficient**.**

1.1. Background of the Organization

Healthcare institutions, including hospitals and clinics, play an important role in providing medical services and handling emergency cases that require blood transfusions. They depend on blood availability and proper coordination to respond to patients' needs.

MEKAKEL is designed to support these institutions by providing a centralized platform for managing blood requests, checking inventory, identifying potential donors, and coordinating with other healthcare facilities.

1.2. General Description of Existing System

In the existing system, blood requests and donor coordination are often handled through phone calls, direct communication, paper records, and separate institutional processes.

Hospitals and clinics may need to contact other healthcare facilities or potential donors manually when the required blood type is unavailable. This approach makes it difficult to access updated information and coordinate requests efficiently.

1.3. Problems of Existing System

The existing blood coordination process faces several challenges:

- Difficulty finding available blood during emergencies.
- Delays in communicating blood requests.
- Lack of centralized blood inventory information.
- Difficulty identifying compatible potential donors.
- Limited coordination between hospitals and clinics.
- Challenges in verifying and managing donor information.
- Lack of an organized platform for tracking blood requests**.**

1.4. Objective of the Project

1.4.1. General Objective

The general objective of MEKAKEL is to develop a web-based blood coordination platform that improves communication and coordination between hospitals, clinics, and volunteer blood donors.

1.4.2. Specific Objectives

The specific objectives of the project are:

- To provide a centralized platform for managing blood requests.
- To allow healthcare institutions to check available blood inventory.
- To identify potential eligible donors based on blood compatibility.
- To improve coordination between hospitals and clinics.
- To provide donor registration and verification processes.
- To enable healthcare institutions to track and manage blood requests.
- To improve communication through notifications.
- To protect user information through role-based access and authorization.

1.5. Overview of the Proposed System

MEKAKEL is a web-based platform that connects hospitals, clinics, and volunteer blood donors. Healthcare institutions can submit blood requests, check available inventory, and identify potential eligible donors when the required blood is unavailable.

The system prioritizes existing blood inventory before searching for potential donors. It also allows healthcare institutions to coordinate with one another and includes donor verification, request management, and notification features.

The platform provides different access levels for donors, ordinary users, hospital staff, doctor-verifiers, and administrators to ensure that each user can access the appropriate features.

1.6. Feasibility Analysis

1.6.1. Economic Feasibility

MEKAKEL is economically feasible because it uses open-source technologies and development tools, reducing software licensing costs. The system can be developed using existing computers and internet access.

Its centralized approach can also reduce the time and communication effort required to coordinate blood requests between healthcare institutions and potential donors.

1.6.2. Technical Feasibility

The project is technically feasible because the required technologies and development tools are available.

The system uses the following technologies:

- Frontend: React, TypeScript, Vite, and Tailwind CSS.
- Backend: Node.js and Express.js.
- Database: PostgreSQL.
- Authentication: JSON Web Tokens (JWT) and bcrypt.
- Development Tools: Visual Studio Code, Git, and GitHub.
- Deployment: Docker.

These technologies support the development of a responsive, secure, and maintainable web-based application.

1.6.3. Operational Feasibility

MEKAKEL is operationally feasible because it is designed with a user-friendly interface and role-based access. Donors, hospital staff, doctor-verifiers, and administrators can use the system according to their responsibilities.

The platform simplifies blood request management, donor coordination, and communication between healthcare institutions. Basic user training may be provided to help users understand and operate the system effectively.

1.7. Relevance of the Proposed System

MEKAKEL is relevant because it addresses coordination challenges in the process of locating and requesting blood. It provides a centralized platform that improves communication between healthcare institutions and potential donors.

The system helps reduce delays in finding available blood, supports better inventory coordination, improves donor information management, and makes blood-related requests easier to track and manage.

1.8. Scope of the Proposed System

The scope of MEKAKEL focuses on blood coordination between hospitals, clinics, and volunteer blood donors.

The system includes:

- User registration, login, and role-based access.

- Hospital and clinic registration and management.
- Blood request creation and management.
- Blood inventory management.
- Blood compatibility-based donor identification.
- Donor registration and document verification.
- Coordination between hospitals and clinics.
- Reservation and donor response management.
- Email and in-app notifications.
- Administrative management and authorization.

The system does not replace blood banks, perform medical blood testing, collect or store physical blood, or make clinical decisions. Blood collection, testing, and transfusion remain the responsibility of authorized healthcare professionals**.**

CHAPTER 2: CONTEXTUAL ANALYSIS AND METHODOLOGICAL FRAMEWORK

2.1. Literature Review

Blood availability and coordination are important parts of healthcare services. According to the World Health Organization (WHO), many patients who require blood transfusions do not receive timely access to safe blood. WHO recommends coordinated blood supply systems to improve the availability, safety, and management of blood.

Digital platforms can help healthcare institutions manage blood inventory, maintain donor information, and improve communication. MEKAKEL follows this approach by providing a centralized platform that connects hospitals, clinics, and volunteer donors.

2.1.1. Experiences of Similar but Failed Projects

Rather than identifying projects as failures without reliable evidence, this section examines common challenges experienced by blood coordination systems.

Some challenges include:

- Outdated blood inventory information.
- Poor communication between healthcare institutions.
- Incomplete donor information.
- Lack of proper donor verification.
- Limited access to centralized information.

- Difficulty maintaining accurate records.

These challenges highlight the importance of developing a reliable, secure, and well-coordinated platform.

MEKAKEL addresses these concerns through centralized information management, donor verification, role-based access, and institutional coordination.

2.1.2. Experiences of Successful Projects

One successful example is **e-RaktKosh**, a centralized blood bank management system developed in India under the Ministry of Health and Family Welfare.

The platform provides services such as:

- Blood availability information.
- Blood bank management.
- Donor registration and management.
- Blood inventory management.
- Blood request services.
- Alerts and notifications.

The system demonstrates how digital technology can improve blood inventory management and connect blood banks with people who need blood.

MEKAKEL adopts similar concepts while focusing on coordination between hospitals, clinics, and volunteer donors. It also includes donor verification and coordination between healthcare institutions.

2.1.3. Benchmarking Considerations

MEKAKEL uses e-RaktKosh as a reference project for understanding how digital blood management systems operate.

The following criteria are considered:

Criteria	**e-RaktKosh**	**MEKAKEL**
Blood inventory	Centralized inventory management	Hospital and clinic inventory management
Donor management	Donor registration and records	Donor registration and verification
Blood requests	Online blood requests	Institutional blood requests
Coordination	Blood bank network	Hospital, clinic, and donor coordination
Notifications	Alerts and notifications	Email and in-app notifications
Technology	Web-based platform	React, Node.js, PostgreSQL
Access	Public and institutional services	Role-based user access

2.2. Methodology

The project follows an iterative software development approach. The system is developed in smaller modules, allowing each module to be implemented, tested, and improved before integration.

This approach helps identify problems early and makes it easier to accommodate changes during development.

2.2.1. Data/Requirement Gathering Methodology

The following methods can be used to identify the system requirements:

1. Document Review

Review existing materials related to blood donation, blood inventory management, and blood request processes to understand the requirements of the proposed system.

2. Interviews

Interviews with potential users, such as healthcare professionals and donors, can help identify their needs, challenges, and expectations.

3. Observation

Observing how blood requests and donor coordination are handled can help identify weaknesses in existing processes.

4. Literature Review

Reviewing research papers, existing blood management platforms, and healthcare guidelines helps identify useful system features and requirements.

2.2.2. Analysis and Modeling Methodology

The project uses both structured and object-oriented modeling techniques.

- **Structured Modeling:** Used to represent data flow, system processes, and database relationships.
- **Object-Oriented Modeling:** Used to represent system actors, objects, interactions, and activities.

The models help developers understand the system before implementation.

2.2.3. Software Tools to Be Used

The following tools support the development and documentation of MEKAKEL.

Tool	Purpose
Visual Studio Code	Code development
Git	Version control
GitHub	Source code management
Draw.io	System diagrams and modeling
Figma	User interface design
PostgreSQL	Database management
Prisma	Database access and modeling
Postman	API testing
Microsoft Word	Project documentation
Docker	Application containerization

CHAPTER 3: REQUIREMENT ANALYSIS AND SPECIFICATION

3.1. Requirement Analysis

Requirement analysis identifies what the proposed system should accomplish and the services it should provide to its users.

MEKAKEL requirements are identified based on the challenges of existing blood coordination processes and the needs of the intended users.

3.1.1. Analysis Approach (BPA/BPI/BPR)

MEKAKEL follows the **Business Process Improvement (BPI)** approach.

BPI focuses on improving existing processes rather than completely replacing them.

The existing blood coordination process often involves manual communication, separate records, and limited information sharing.

MEKAKEL improves this process by introducing:

- Centralized blood request management.
- Digital blood inventory management.
- Donor registration and verification.
- Coordination between healthcare institutions.
- Automated notifications.
- Organized request tracking.

This approach improves existing coordination activities while allowing healthcare institutions to continue their normal medical operations.

3.1.2. Functional Requirements

Functional requirements describe the services and operations provided by the system.

MEKAKEL has the following functional requirements:

1. The system shall allow users to register and log in.
2. The system shall support different user roles.
3. The system shall allow donors to create and manage their profiles.
4. The system shall allow hospitals and clinics to manage their institutional profiles.
5. The system shall allow authorized hospital staff to create blood requests.
6. The system shall allow healthcare institutions to manage blood inventory.
7. The system shall identify potentially compatible donors based on blood group information.
8. The system shall prioritize available blood inventory before searching for potential donors.
9. The system shall allow hospitals and clinics to coordinate blood requests.
10. The system shall allow donors to accept or decline donation requests.
11. The system shall support donor document submission and verification.
12. The system shall allow authorized doctors to review donor documents.
13. The system shall provide email and in-app notifications.
14. The system shall allow administrators to manage users and system activities.
15. The system shall support reservation and request status management.

3.1.3. Nonfunctional Requirements

Nonfunctional requirements describe the quality and performance expectations of the system.

1. Security

The system should protect user information through authentication, authorization, password hashing, and role-based access control.

2. Performance

The system should respond to user requests efficiently and retrieve relevant information within a reasonable time.

3. Usability

The system should provide a simple and understandable interface that users can navigate easily.

4. Reliability

The system should maintain accurate records and handle system errors without unnecessary data loss.

5. Scalability

The system should support future growth in users, healthcare institutions, and system activities.

6. Maintainability

The system should have an organized structure that allows developers to update, test, and maintain its components.

7. Availability

The system should be accessible to authorized users whenever the hosting infrastructure and network are available.

8. Compatibility

The system should work with modern web browsers and different screen sizes.

3.2. Comparative Analysis and Solution Proposal

This section compares the existing blood coordination process with the proposed MEKAKEL system.

3.2.1. Benchmarking Analysis

The e-RaktKosh platform demonstrates the importance of centralized blood inventory management and donor coordination.

MEKAKEL uses similar concepts but focuses on connecting hospitals, clinics, and volunteer donors through a coordinated request management process.

The following features are considered important:

- Centralized information management.
- Blood inventory tracking.
- Donor management.
- Blood request processing.
- Institutional communication.
- Secure access to information.

3.2.2. Gap Analysis

Gap analysis identifies differences between the existing process and the proposed system.

Existing System	Identified Gap	MEKAKEL Solution
Manual communication	Delayed information exchange	Centralized communication
Separate inventory records	Difficult to check availability	Digital inventory management
Manual donor searching	Time-consuming coordination	Compatibility-based donor identification
Limited institutional coordination	Difficulty locating blood elsewhere	Hospital and clinic coordination
Paper-based donor information	Difficult record management	Digital donor profiles
Manual request tracking	Limited request visibility	Request status tracking
Limited verification processes	Difficult to manage donor records	Document submission and verification

3.2.3. Proposed Solution Based on Requirements

MEKAKEL proposes a centralized web-based platform to improve blood coordination.

The system addresses the identified gaps through the following solutions:

- **Inventory Management:** Healthcare institutions can manage and check available blood inventory.
- **Donor Identification:** The system identifies potential donors based on blood group compatibility.

- **Institutional Coordination:** Hospitals and clinics can coordinate when the required blood is unavailable at their facilities.
- **Donor Verification:** Donors can submit supporting documents for review by authorized verifiers.
- **Request Management:** Healthcare institutions can create and track blood requests.
- **Notifications:** Users receive relevant updates through email and in-app notifications.
- **Access Control:** Different user roles have access to features according to their responsibilities.

These features improve coordination, information management, and communication without replacing professional blood banks or medical procedures.

3.3. Requirement Modeling

Requirement modeling represents the system requirements using diagrams and structured representations.

MEKAKEL uses structured and object-oriented modeling techniques.

3.3.1. Structured Requirement Modeling

Structured modeling focuses on the flow of information and the processes performed by the system.

The following diagrams can be included:

- **Context Diagram:** Shows MEKAKEL and its external entities, including donors, hospitals, clinics, and administrators.
- **Data Flow Diagram (DFD):** Shows how information moves between users, processes, and the database.
- **Entity Relationship Diagram (ERD):** Shows database entities and their relationships.
- **Flowcharts:** Represent important processes such as registration, blood requests, donor verification, and inventory checking.

3.3.2. Object-Oriented Requirement Modeling

Object-oriented modeling represents the system through objects, actors, and their interactions.

The following UML diagrams are recommended:

- **Use Case Diagram:** Shows system actors and their interactions with MEKAKEL.
- **Class Diagram:** Represents system classes, attributes, methods, and relationships.
- **Sequence Diagram:** Shows interactions between users and system components over time.

- **Activity Diagram:** Represents workflows such as creating blood requests and verifying donors.

CHAPTER 4: SYSTEM DESIGN

4.1. System Architecture

MEKAKEL follows a three-tier architecture consisting of the presentation layer, application layer, and database layer.

1. Presentation Layer

This layer provides the user interface through which donors, hospitals, clinics, and administrators interact with the system.

Technologies:

- React
- TypeScript
- Vite
- Tailwind CSS

2. Application Layer

This layer handles business logic, authentication, authorization, request processing, donor identification, and communication.

Technologies:

- Node.js
- Express.js
- TypeScript
- Prisma
- JWT

3. Database Layer

This layer stores and manages system information, including user accounts, donor profiles, institutional information, blood inventory, reservations, and notifications.

Technology:

- PostgreSQL

The frontend communicates with the backend through APIs, while the backend communicates with the database to retrieve and manage information.

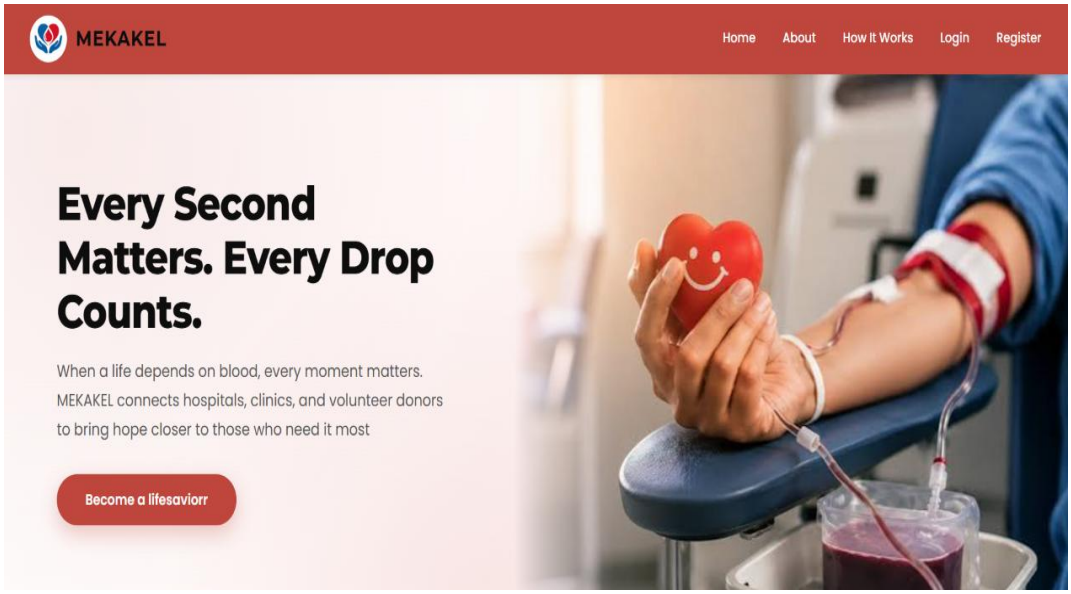
4.2. Interface Design

The MEKAKEL interface is designed to be simple, responsive, and user-friendly.

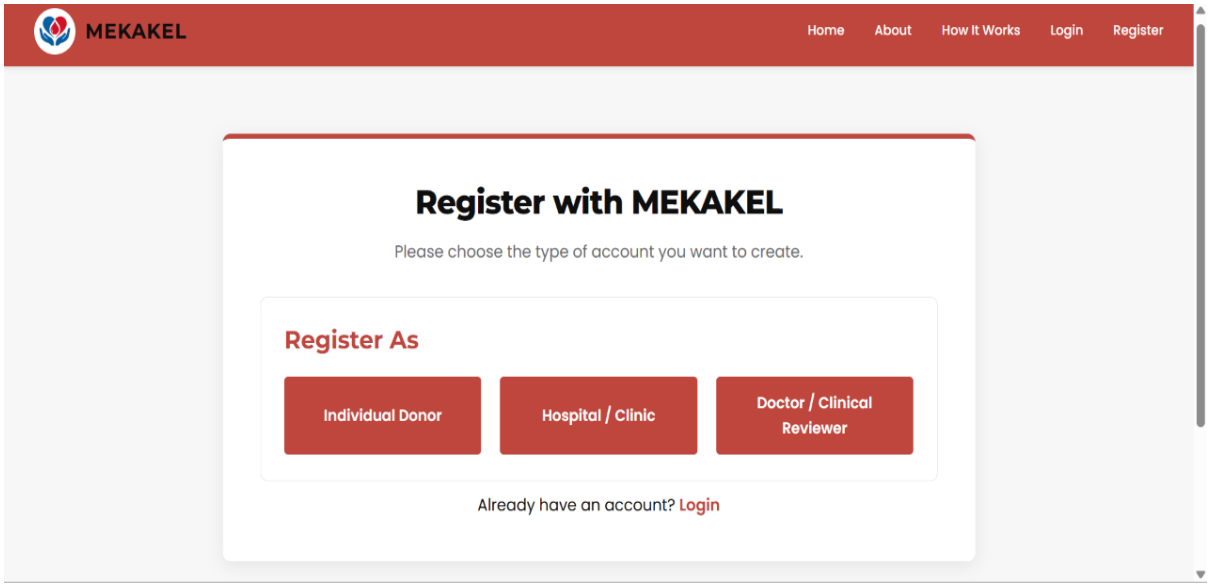
The system provides different interfaces based on user roles.

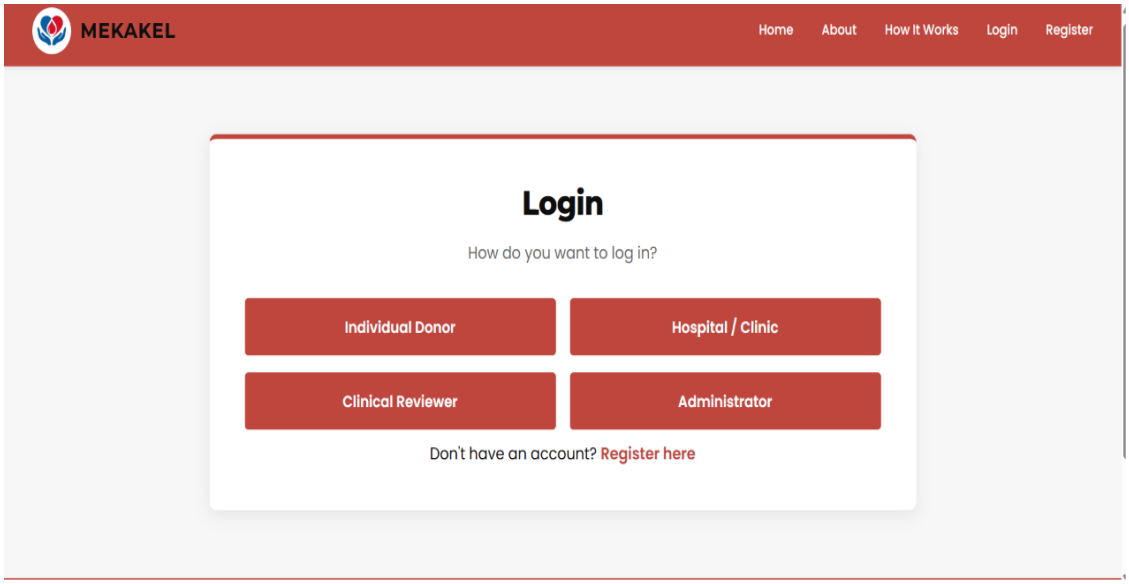
The main interfaces include:

- **Landing page.**

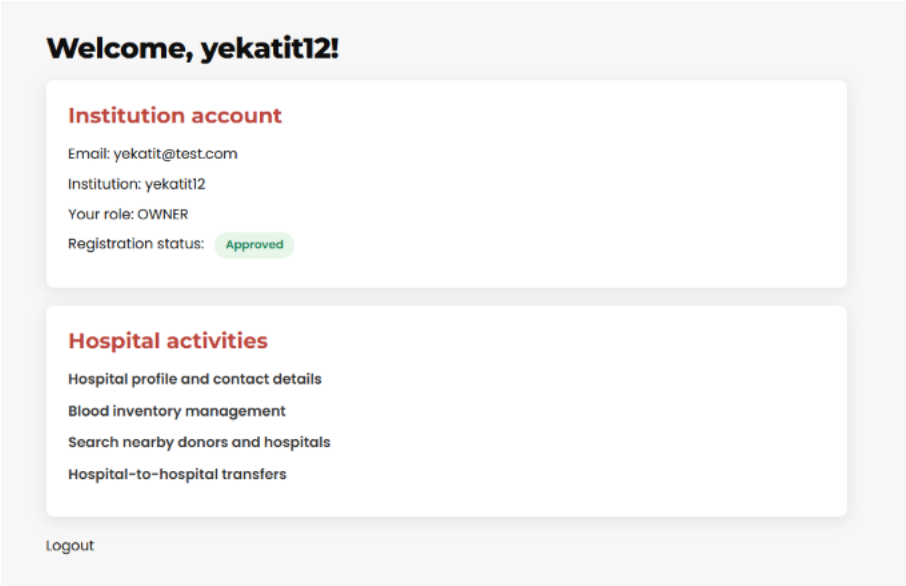


- **Registration and login pages.**

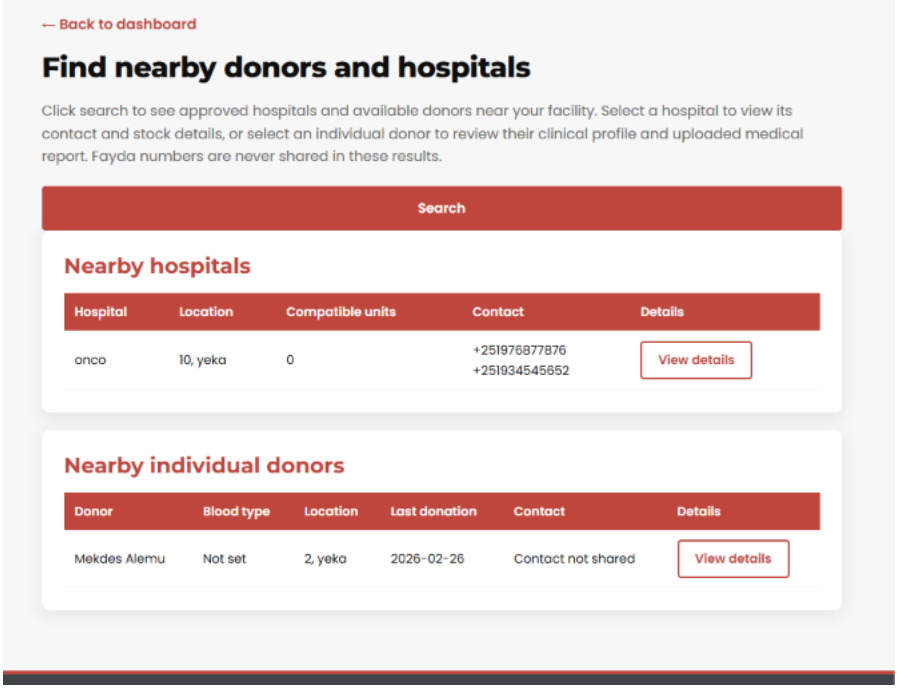




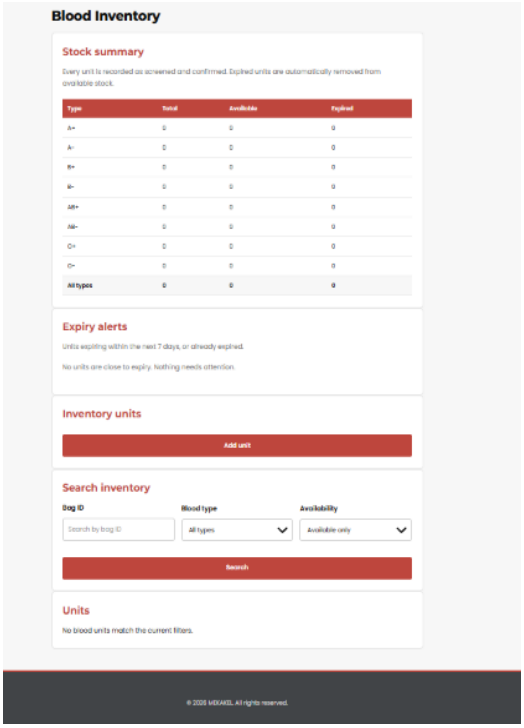
- **Donor dashboard.**
- **Hospital and clinic dashboards.**



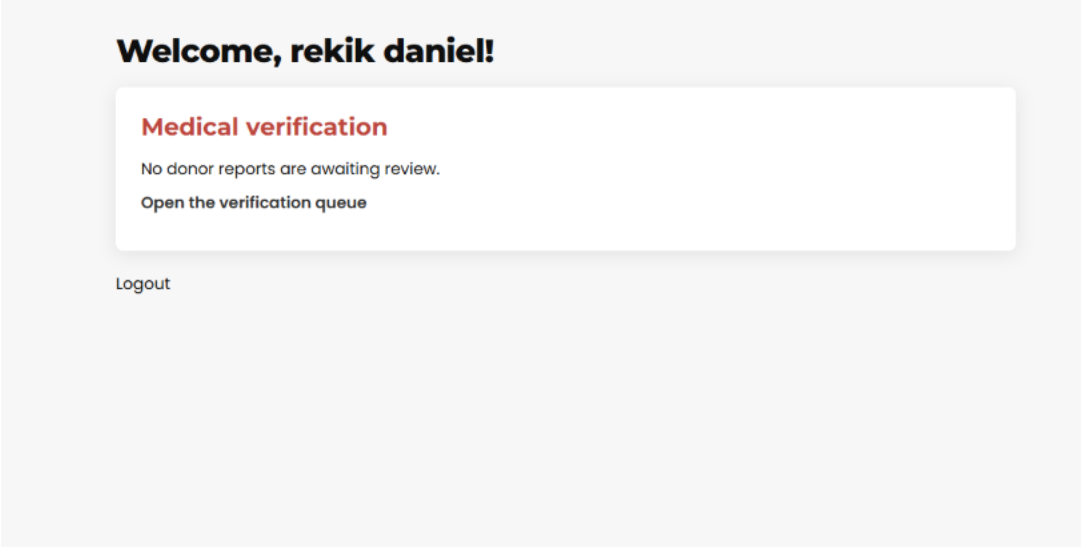
- **Blood request management page.**



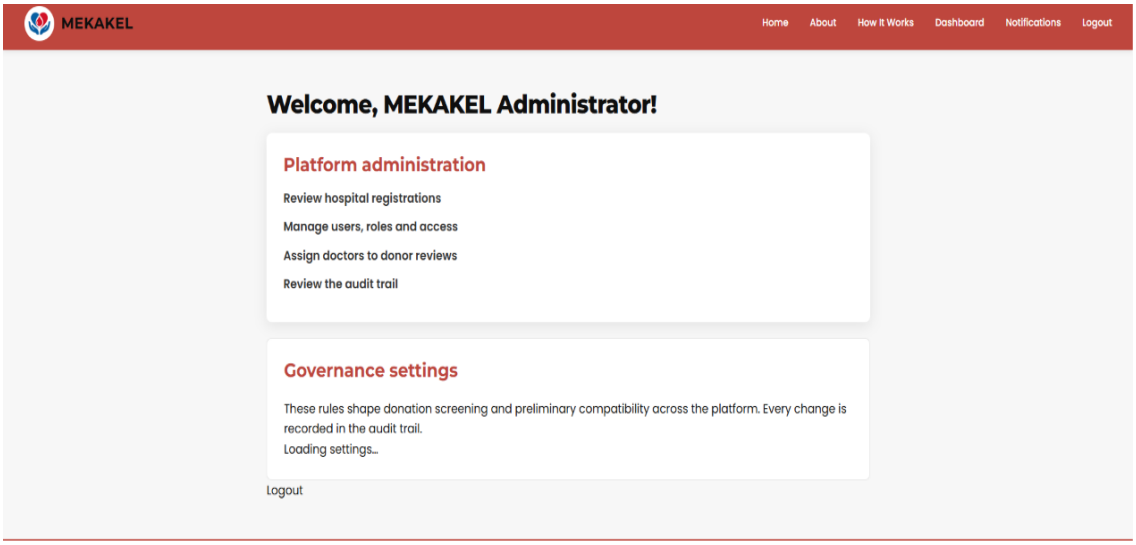
- **Blood inventory management page**.

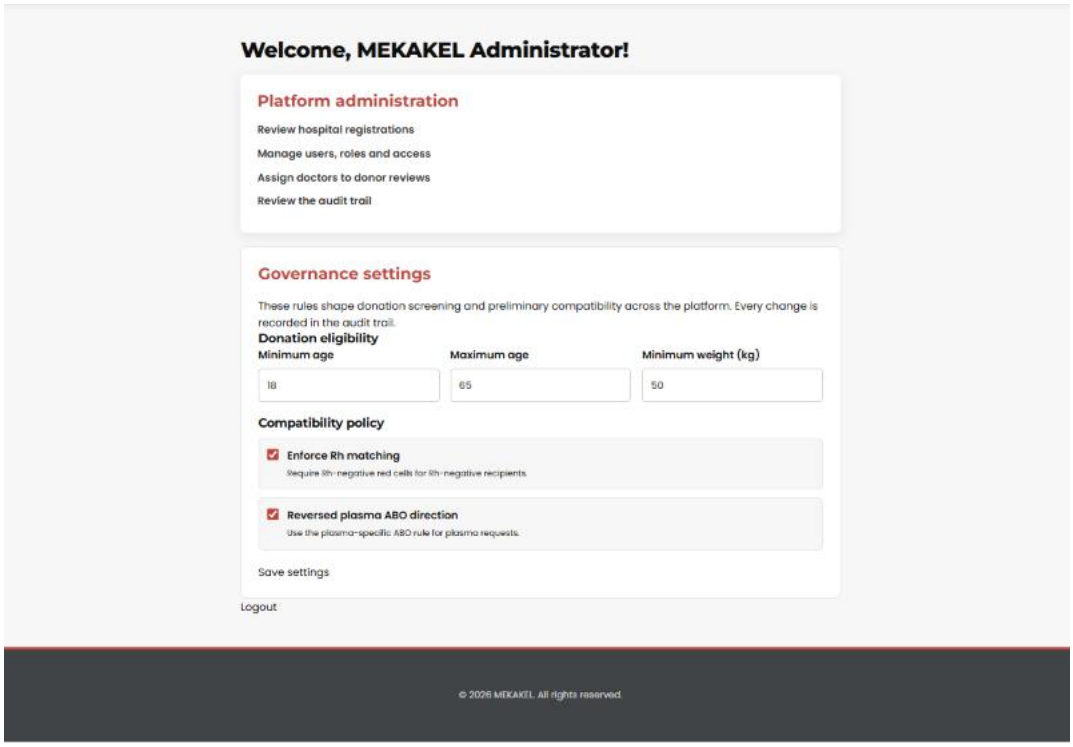


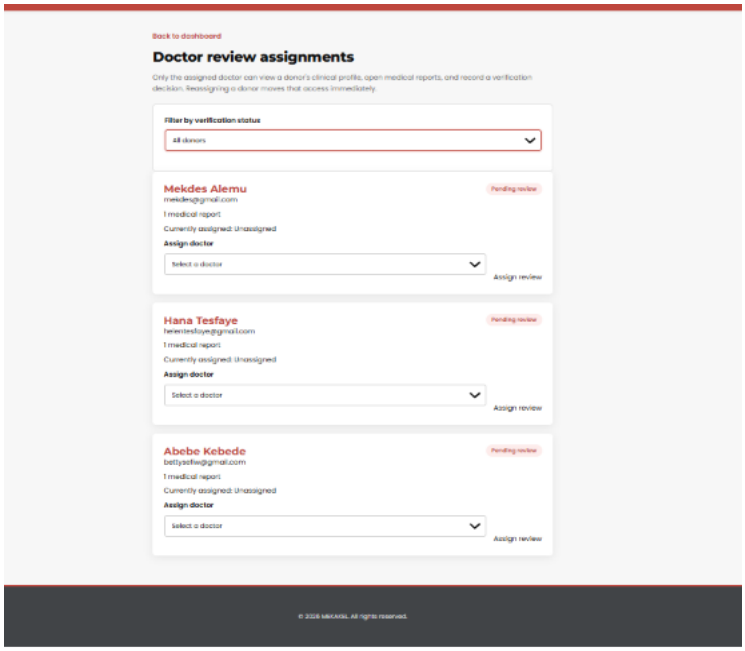
- Donor profile and verification page.
- Doctor-verifier dashboard.



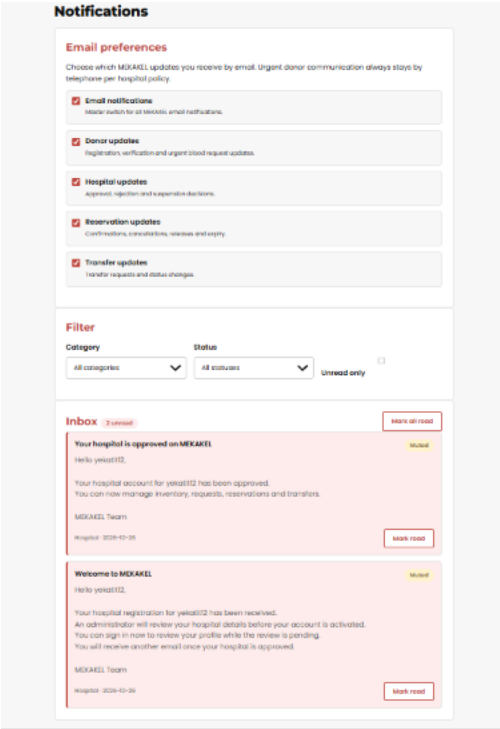
- Administrator dashboard.







- Notification interface.



4.3. Database/Data Structure Design

MEKAKEL uses PostgreSQL as its relational database management system.

The database is designed to store and organize information about users, healthcare institutions, donors, blood inventory, requests, reservations, and notifications.

The major database entities include:

Entity	Description
User	Stores user account information and roles.
Donor	Stores donor profile information.
Hospital	Stores hospital information.
Clinic	Stores clinic information.
BloodUnit	Stores available blood inventory records.

Entity	Description
BloodRequest	Stores blood requests created by institutions.
Reservation	Stores reservation and donor response information.
DonorReport	Stores submitted donor documents and verification status.
Notification	Stores user notifications.

Relationships between these entities help maintain organized and consistent information.

An Entity Relationship Diagram (ERD) should be included to illustrate the database structure.

4.4. Algorithm/Program Design

This section explains the major algorithms and processes used in MEKAKEL.

1. User Authentication Algorithm

- Receive login credentials.
- Validate the submitted information.
- Verify the user's password.
- Generate an authentication token.
- Allow access according to the user's role.

2. Blood Request Processing Algorithm

- Hospital staff create a blood request.
- The system checks available blood inventory.
- If suitable inventory is available, the system identifies the relevant units.
- If inventory is unavailable, the system searches for potential eligible donors.
- The request is processed according to the applicable workflow.

3. Donor Identification Algorithm

- Retrieve donor records.
- Check blood group compatibility.
- Consider donor eligibility information.
- Identify potential matching donors.

- Present eligible candidates for coordination.

4. Donor Verification Algorithm

- Donor submits supporting documents.
- The system stores the submitted documents.
- An authorized doctor reviews the documents.
- The doctor approves or rejects the submission.
- The system updates the verification status.

5. Reservation Management Algorithm

- A donor receives a donation request.
- The donor accepts or declines.
- The system updates the reservation status.
- Relevant users receive notifications.

These algorithms support the main operations of the MEKAKEL platform.

CHAPTER 5: RISK MITIGATION STRATEGY

Risk management identifies possible problems that may affect the development, security, and operation of the system.

5.1. Risk Identification

The following risks may affect MEKAKEL:

1. Unauthorized access to user accounts.
2. Exposure of confidential donor information.
3. Incorrect or outdated blood inventory information.
4. Network or server failures.
5. Incorrect donor information.
6. Database failures or data loss.
7. Delays in donor verification.
8. Low user adoption.

9. Incorrect system configuration during deployment.

5.2. Risk Assessment

Risks are evaluated according to their likelihood and potential impact.

Risk	Likelihood	Impact	Risk Level
Unauthorized access	Medium	High	High
Data exposure	Medium	High	High
Incorrect inventory information	High	High	High
Server failure	Medium	High	High
Incorrect donor information	Medium	Medium	Medium
Database failure	Low	High	Medium
Verification delays	Medium	Medium	Medium
Low user adoption	Medium	Medium	Medium

5.3. Mitigation Strategies

The following strategies are proposed to reduce identified risks.

Risk	Mitigation Strategy
Unauthorized access	Implement authentication and role-based authorization.
Data exposure	Protect sensitive information and restrict access.
Incorrect inventory	Provide inventory update and management features.
Server failure	Use monitoring and recovery procedures.
Incorrect donor information	Introduce donor verification processes.
Database failure	Perform regular database backups.

Risk	Mitigation Strategy
Verification delays	Provide organized verification workflows.
Low user adoption	Design a simple interface and provide user guidance.
Deployment problems	Test deployment configurations before production.

CHAPTER 6: CONCLUSION AND RECOMMENDATIONS

6.1. Conclusion

MEKAKEL is a web-based blood coordination platform developed to improve communication and coordination between hospitals, clinics, and volunteer blood donors.

The system addresses challenges such as limited access to blood inventory information, difficulties in identifying potential donors, and delays in coordinating blood requests.

By providing centralized inventory management, donor identification, verification, request tracking, and institutional coordination, MEKAKEL aims to make blood-related coordination more organized and efficient.

The platform supports existing healthcare services and does not replace professional blood banks, medical testing, or clinical decision-making.

6.2. Recommendations

The following recommendations are proposed for the future improvement of MEKAKEL:

1. Integrate the system with more healthcare institutions.
2. Introduce SMS and USSD notifications to support users with limited internet access.
3. Integrate with authorized digital identity verification services.
4. Improve geographical search to help locate nearby healthcare institutions and potential donors.
5. Introduce additional reporting and analytics features.
6. Improve integration with existing healthcare information systems.
7. Conduct regular security testing and system maintenance.

6.3. Future Enhancements

Future versions of MEKAKEL may include the following improvements:

1. **SMS and USSD Integration:** Enable users to receive notifications and access essential services without an internet connection.
2. **Fayda Integration:** Integrate with Ethiopia's Fayda digital identity system for improved identity verification.
3. **Mobile Application:** Develop Android and iOS applications for easier access.
4. **Advanced Analytics:** Introduce dashboards and reports to analyze blood demand, availability, and donation activities.
5. **Geographical Integration:** Improve location-based searching to identify nearby hospitals, clinics, and potential donors.
6. **Healthcare System Integration:** Connect MEKAKEL with existing hospital information systems.
7. **Multilingual Support:** Support multiple Ethiopian languages to make the platform more accessible.

6.4. Plan for System Construction, Testing, Conversion, and Support

The development and implementation of MEKAKEL follow several stages.

1. System Construction

The system is developed using React, TypeScript, Node.js, Express.js, and PostgreSQL. Frontend and backend modules are implemented and integrated.

2. System Testing

Different testing methods are used to verify system functionality and reliability.

- Unit Testing: Tests individual functions and components.
- Integration Testing: Checks communication between system modules.
- API Testing: Verifies backend endpoints.
- Authentication Testing: Checks login, permissions, and access control.
- User Interface Testing: Checks usability and responsiveness.

3. System Conversion

The system is prepared for deployment and introduced to its intended users. Existing information can be registered in the platform where appropriate, following proper verification and authorization.

4. System Support and Maintenance

After deployment, the system requires continuous support to maintain its performance and reliability.

Maintenance activities include:

- Fixing identified errors.
- Updating system features.
- Monitoring application performance.
- Performing database backups.
- Applying security updates.
- Providing technical support to users.

REFERENCES

The following references support the literature review and technical aspects of MEKAKEL. They are formatted using APA style.

1. World Health Organization. (2026). *Blood safety and availability*. WHO Blood Safety and Availability
2. Ministry of Health and Family Welfare, Government of India. (n.d.). *e-RaktKosh: A centralized blood bank management system*. e-RaktKosh Official Website
3. Centre for Development of Advanced Computing. (n.d.). *e-RaktKosh: One Nation One Platform for information on blood centers and blood bank management*. C-DAC e-RaktKosh Project
4. World Health Organization. (2026). *Global status report on blood safety and availability 2025*. WHO Global Status Report
5. World Health Organization. (n.d.). *Blood cold chain*. WHO Blood Cold Chain